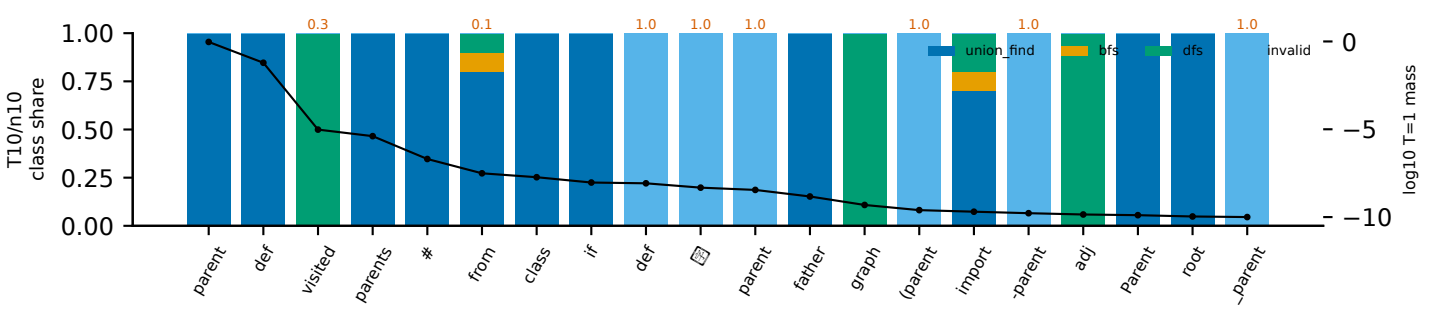
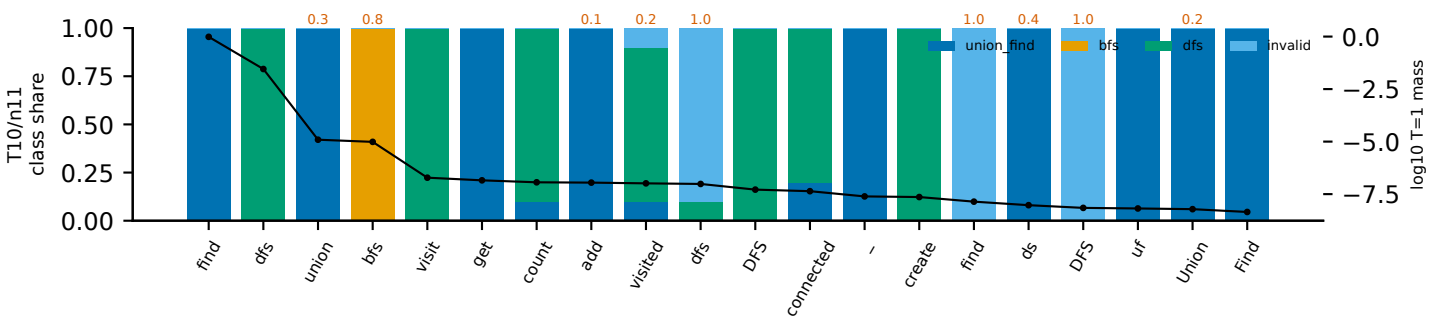
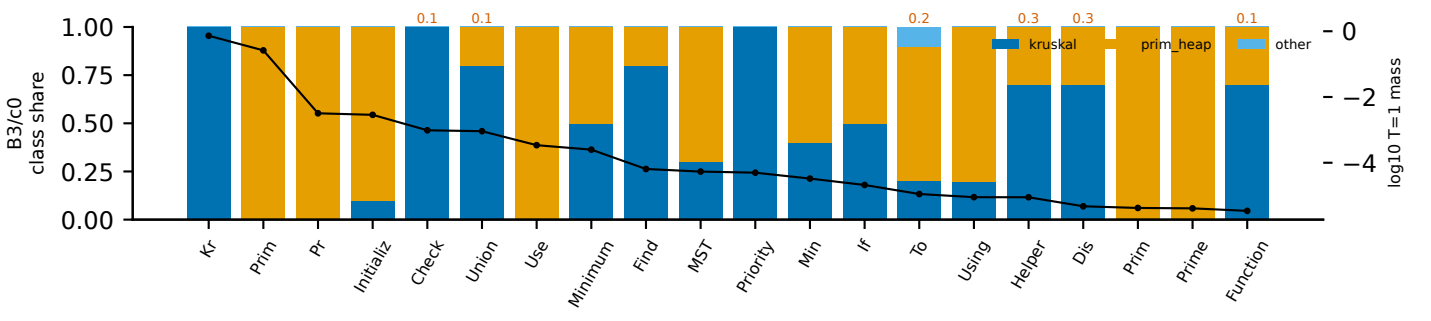
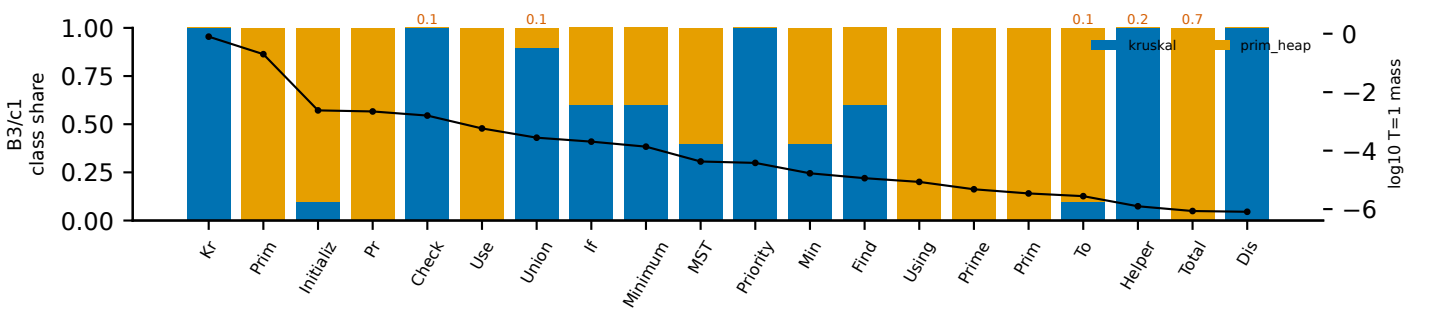
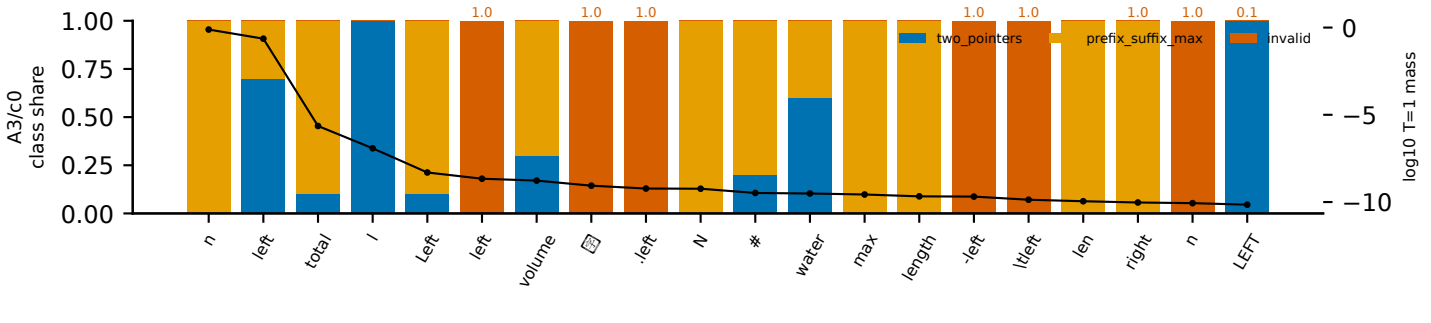
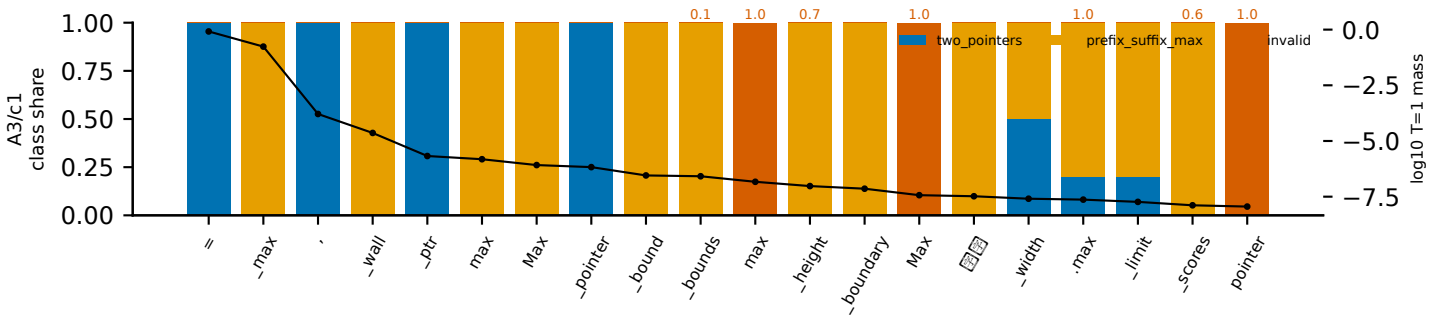

T10/n10
class share
log10 T=1 mass
union_find
bfs
dfs
invalid
parent def visited parents # from class if def [ parent father graph (parent import -parent adj Parent root _parent

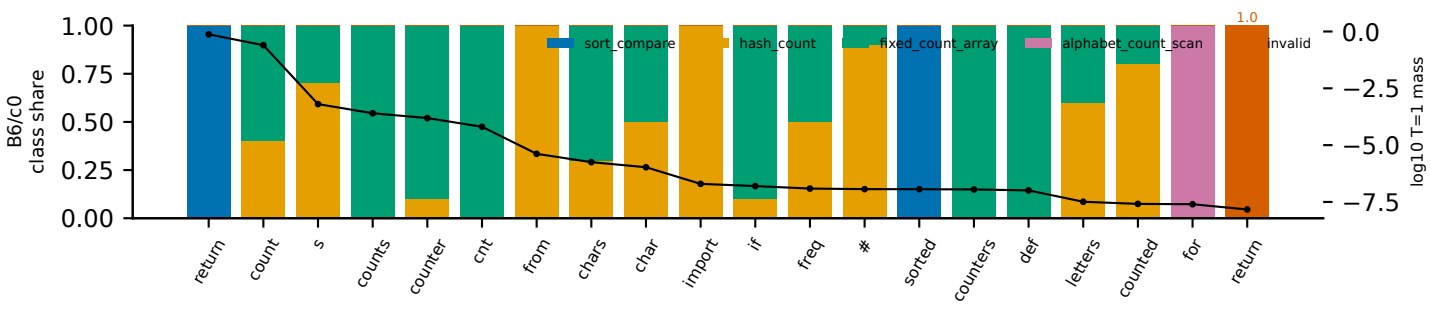

T10/n11
class share
log10 T=1 mass
union_find
bfs
dfs
invalid
find dfs union bfs visit get count add visited dfs DFS connected _ create find ds DFS uf Union Find

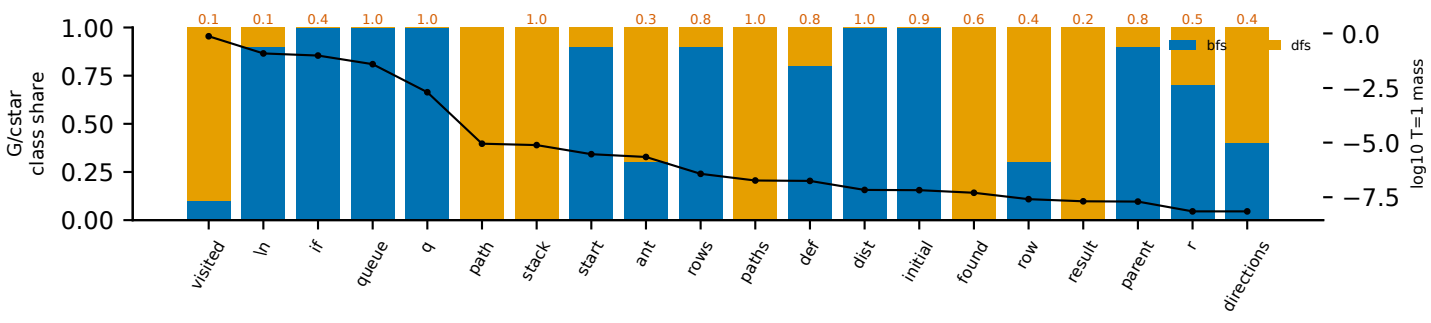

B3/c0
class share
log10 T=1 mass
kruskal
prim_heap
other
Kr Prim Pr Initializ Check Union Use Minimum Find MST Priority Min If To Using Helper Dis Prim Prime Function


B3/c1
class share
log10 T=1 mass
kruskal
prim_heap
Kr Prim Initializ Pr Check Use Union If Minimum MST Priority Min Find Using Prime Prim To Helper Total Dis


A3/c0
class share
log10 T=1 mass
two_pointers
prefix_suffix_max
invalid
n left total l Left left volume [ .left N # water max length -left ltleft len right n LEFT


A3/c1
class share
log10 T=1 mass
two_pointers
prefix_suffix_max
invalid
= _max , _wall _ptr max Max _pointer _bound _bounds max _height _boundary Max [ _width .max _limit _scores pointer


B6/c0
class share
log10 T=1 mass
sort_compare
hash_count
fixed_count_array
alphabet_count_scan
invalid
return count s counts counter cnt from chars char import if freq # sorted counters def letters counted for return


G/cstar
class share
log10 T=1 mass
bfs
dfs
visited ln if queue q path stack start ant rows paths def dist initial found row result parent r directions